

# Second Brain — Design Overview

Working model as agreed through decision 23 · maintained document, enriched as decisions land

## PREMISE

What the system is

You dictate unstructured thoughts — mostly takeaways from podcasts, blogposts and meetings. An agentic pipeline turns them into a structured, linked, searchable knowledge base. Everything is markdown in a Git repository, so the knowledge outlives the app.

**The organising idea: every artifact is either *derived* or *decided*.** Derived artifacts are pipeline-owned, disposable and rebuilt from scratch at will. Decided artifacts are human-owned, durable and never regenerated. Nothing is both. Almost every rule below follows from that one split.

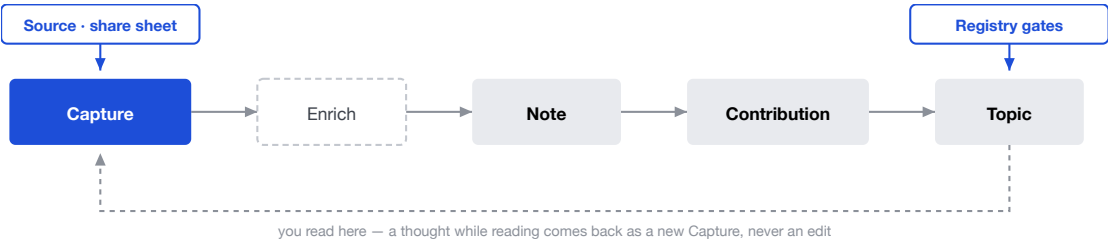
## ARTIFACTS

The vocabulary

| TERM         | WHAT IT IS                                                          | WRITER  | IDENTITY FROM  |
|--------------|---------------------------------------------------------------------|---------|----------------|
| Capture      | A raw thought as you recorded it, unedited by the system            | decided | Timestamp      |
| Task         | An Item you accepted — durable work, outlives the Note it came from | decided | Its own        |
| Registry     | Canonical entity names and their aliases — the naming authority     | decided | You            |
| Note         | A cleaned, tagged, linked document derived from one Capture         | derived | Its Capture    |
| Contribution | The few claims one Note makes about one Topic — the cached unit     | derived | Note + Topic   |
| Topic        | A synthesis of what you know about one entity, plus a link index    | derived | Registry name  |
| Source       | Where a Capture came from: podcast, blogpost, book, meeting         | derived | URL (external) |
| Candidate    | An entity the pipeline noticed that you haven't admitted yet        | derived | —              |
| Kind         | What sort of Capture a Note came from — one per Note, closed set    | derived | —              |
| Item         | A proposed task, question or decision — promote it to make it real  | derived | —              |

## FLOW

Capture to knowledge



**Nightly:** new Captures sync → Notes derived → Contributions extracted and cached → only Topics whose Contribution set actually changed are re-synthesised. Cost scales with what you captured that day, never with the size of the corpus.

## INVARIANTS

The load-bearing rules

- **One writer per artifact.** You write Captures and the Registry; the pipeline writes everything else. Conflicts are impossible rather than resolved.
- **Nothing invents an identifier.** A Note inherits its Capture's identity, a Topic its Registry name, a Source its URL. Anything an LLM would have to name freely is forbidden, because it drifts between runs and rots every link.
- **Never guess, leave it unresolved.** A wrong entity match or wrong source URL is silent and permanent; an unresolved mention is visible and harmless.
- **Decided state never lives on a derived artifact.** Anything you choose — a promoted entity, an accepted task — needs a durable home with identity of its own, or the next replay destroys it.
- **Derived artifacts are disposable.** Delete and replay at any time. Improving the pipeline is a replay, never a migration.
- **Staleness is a hash, not a timestamp.** A timestamp records when work happened; only a content hash plus pipeline version says whether it needs doing again.
- **You never hand-edit a derived artifact.** Drift detection catches it and refuses to overwrite rather than destroying the edit.

## DECISIONS

docs/adr/

- 01 Git is the system of record.** Not GCS (no history), not a database (markdown becomes second-class). Everything computed is a disposable index.
- 02 Captures and Notes are separate, single-writer artifacts.** Lets the pipeline freely rewrite dictated prose while guaranteeing your original words are never altered.

- 03 **Notes are read-only, enforced by drift detection.** An unenforced invariant is only a hope — Obsidian will happily let you edit one.
- 04 **Derived documents never invent identifiers.** Notes are 1:1 with Captures; atomicity arrives at the Topic layer. Collapses the PRD's "auto-generated Wiki" into the pipeline — a wiki page *is* a Topic.
- 05 **You are the sole naming authority for Topics.** The pipeline raises Candidates; you promote them. Canonical naming is a judgement, and an LLM making it yields unstable identity.
- 06 **Tags and Topics are separate vocabularies.** Subjects grow without limit and are Registry-governed; facets are a small closed set the pipeline may never extend. Conflating them is what causes tag sprawl.
- 07 **Topics are flat files, hierarchy is metadata.** Membership is many-to-many and hierarchy is the most volatile structure you have — folders would make every change of mind a migration.
- 08 **Sources bypass the Registry.** A URL is assigned by the world, not chosen, so nothing is invented. Keeps the review list small enough that you'll actually work it.
- 09 **Provenance is supplied at capture, never inferred.** Via the OS share sheet — which makes the mobile app a *share target*, not a capture app. Missing provenance is marked `unresolved`, never silently self-attributed.
- 10 **Topic synthesis is map-reduce over cached Contributions.** LLM work sits at the Note level where it is 1:1 and cacheable; Topic assembly stays mechanical. Without this, the 3am job gets slower every week until it stops finishing.
- 11 **Every artifact is either derived or decided.** The principle the earlier decisions were circling. The combination with no home is decided state inside a derived artifact — exactly what a checked-off todo would be.
- 12 **Items are proposals; accepting one promotes it to a Task.** Reuses the Candidate→Registry pattern instead of a third mechanism. Promotion doubles as triage, and a Task survives replay because decided state is never regenerated.
- 13 **Compute is split by workload.** Pipeline as a Cloud Run Job; the app ships its read index to the browser and the server only commits decided state. Semantic search lives in the pipeline, so embeddings never reach the client.
- 14 **GitHub hosts the Vault.** Google has no general-purpose Git host any more — the one deliberate non-Google dependency, and it brings Actions as a fallback runner.
- 15 **The Vault is an OKF bundle.** Google's Open Knowledge Format is real and permissive — one required key, arbitrary extras preserved. Links become standard markdown rather than wikilinks, which costs nothing because only the pipeline ever writes a link.
- 16 **The domain owns frontmatter; the adapter owns the file.** Typed rules over the fields that carry them, unknown keys passed through opaquely, the body opaque to the domain. Paths become an adapter concern, which is what makes renaming cheap.
- 17 **The system boundary starts at text.** The transcript is the Capture, so audio never enters Git and typing is a first-class input rather than a fallback. Whisper runs locally and is vocabulary-biased with the Registry, so the names you use most transcribe correctly.
- 18 **The MVP has no app.** Obsidian is the whole interface; the review loops are just two files you move a line between. The risk lives in the pipeline, so that is where the first release spends its effort.
- 19 **The harness guards determinism, not just correctness.** Cassettes make LLM calls testable, evals answer "did it get worse" separately from "did it break", and a golden-corpus replay must be byte-identical.
- 20 **Backlinks are computed, and the graph is bipartite.** Storing them would make every new Capture rewrite old Notes. No Note→Note links — connection runs through a named Topic, so every edge is explainable. Similarity proposes Candidates rather than guessing edges.
- 21 **No contribution graph.** A streak counter rewards volume of Captures and would corrupt the corpus it measures. Dormancy — what has gone quiet, what is piling up unreviewed — is the signal worth surfacing.
- 22 **No LLM-generated HTML views.** Enrichment reads untrusted web content, so generated markup is a path from a hostile page to script execution inside a PWA holding the whole knowledge base and a Git credential.
- 23 **The Vault is provisioned by an init command.** A separate private repository, created and protected by tooling rather than by hand — under a bootstrap credential the operator holds locally, never the runtime one that reaches Cloud Run.

---

## OPEN

Next up

? **Next step.** Slice 1 — one Capture in Git, through a Cloud Run Job, back as a committed Note.